

SPOTTER

BE SPOT ON.

Architecture & Design Document

AI-Powered Smart Gym System

A localized, bilingual smart-fitness platform: on-device posture AI, an affordable ESP32 wearable, and Egyptian-first nutrition guidance.

Item	Detail
Document	Architecture & Design Document (ADD)
Date	October 2026
Prepared by	Abdelaziz Emara & Hala Elbialy
Supervisor	Dr. Hania Hassan
Institution	Alexandria National University — Computer & Communications Engineering
Companion documents	Requirements Engineering (SRS); Firestore Schema Diagram

Contents

1. Introduction....3
2. Architectural Drivers....3
3. Architecture Overview....4
  3.1 Architectural Style....4
  3.2 Layer Responsibilities....5
4. Component Design....5
5. Deployment View....6
6. Key Runtime Scenarios....7
  6.1 Live Workout Set....7
  6.2 Chatbot Message with Failover....8
  6.3 Offline Training and Reconnection....8
7. Pose-Coaching Design....9
  7.1 Pipeline....9
  7.2 Model Selection....9
8. Wristband Design....9
9. Cloud Services and AI....10
  9.1 Firebase Instead of a Backend....10
  9.2 Chatbot....10
  9.3 Semantic Search....10
  9.4 Meal Planner....10
10. Data Design....11
  10.1 Design Rules....11
  10.2 Capacity Check Against the Free Tier....11
11. Security, Privacy and Localisation....12
12. Key Design Decisions....12
13. Risks and Mitigations....13
14. References....14
Appendix A. Traceability Matrix....15

1. Introduction

This document describes **how** Spotter, the AI-Powered Smart Gym System, is built. The Requirements Engineering document (SRS) states what the system must do; this document explains the architecture that delivers it — the main parts of the system, how they communicate, where data lives, and why the key design choices were made.

Scope: the Flutter (Dart) mobile app with its on-device pose-coaching pipeline; the ESP32 wristband and its Bluetooth Low Energy (BLE) link; and the Firebase services the app uses directly — Auth, App Check, Cloud Firestore, Firebase AI Logic (Gemini), Firestore Vector Search and Remote Config. There is **no custom backend server**.

Term	Meaning
BLE	Bluetooth Low Energy — the low-power link between wristband and phone
BlazePose	Google's on-device pose model that finds 33 body landmarks in each camera frame [2], used through the pose_detection package [4]
Firebase AI Logic	Firebase service that lets the app call Gemini without holding an API key
App Check	Firebase service that confirms requests come from the genuine app
Embedding	A numeric vector representing meaning, used for semantic search
TDEE	Total daily energy expenditure — the user's daily calorie target

Table 1: Key terms

2. Architectural Drivers

A few requirements shape the architecture more than all the others. Every major design decision in this document follows from one of them.

Driver	Source	Consequence for the design
Real-time posture feedback	NFR-PERF-01/02	Pose AI and feedback run on the phone — no network round trip.
Camera privacy	FR-CV-11, NFR-SEC-04	No frame or image ever leaves the device; only per-set summaries are stored.
Gyms have poor connectivity	NFR-REL-02, C-ENV-05	Offline-first app: Firestore offline persistence and a local Isar catalogue.
Firestore is the only database	C-TECH-02	Denormalised document model; semantic search also inside Firestore.
Free-tier budget	NFR-SCAL-02	Biometrics aggregated before writing; LLM calls rate-limited.
No secrets in the app	NFR-SEC-05	Gemini reached through Firebase AI Logic, which holds the key.

Table 2: Architectural drivers

3. Architecture Overview

3.1 Architectural Style

Spotter follows a **client-heavy, offline-first, serverless architecture**. All application logic is written in **Dart** inside the Flutter app: pose estimation, rep counting, posture scoring, nutrition calculations, meal planning, alerts and chatbot prompt building. The app talks **directly to Firebase**, which provides the cloud side as managed services. There is no custom backend to build, host or secure.

Inside the app a strict **three-layer architecture** is used (NFR-MNT-01): presentation → domain → data. The domain layer is pure Dart, so it can be unit-tested without a phone, camera or Firestore.

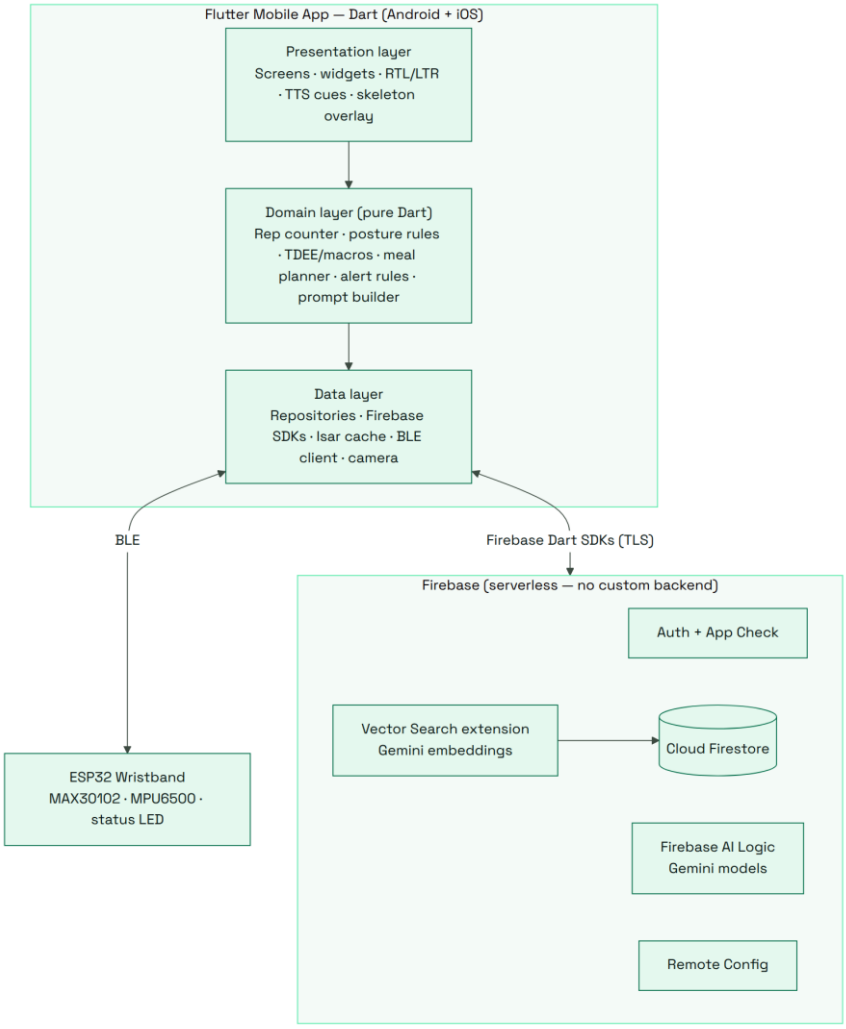


Figure 1: Layered architecture and external systems

3.2 Layer Responsibilities

Layer / tier	Responsibilities	Key technologies
Presentation (app)	Screens, Arabic/English RTL layout, camera preview with skeleton overlay, spoken cues, charts, chat	Flutter widgets, flutter_tts
Domain (app)	Joint angles, rep counting, posture rules, form score, BMI/TDEE/macros [13], HR zones [14] and alerts, meal planner, chat prompt builder	Pure Dart
Data (app)	Repositories hiding Firestore, Isar [26], BLE, camera and Firebase AI	cloud_firestore, firebase_ai, isar, flutter_blue_plus, pose_detection
Wristband	Heart rate, steps and calorie estimate on the band; battery protection; BLE updates every 2 s	ESP32, MAX30102, MPU6500
Firebase (cloud)	Identity, storage and sync, security rules, LLM access, vector search, remote settings	Auth, App Check, Firestore, AI Logic, Vector Search, Remote Config

Table 3: Layer responsibilities

4. Component Design

Figure 2 shows the main software components and how they communicate. On the phone, the camera → pose engine → rule engine → feedback chain is the real-time coaching loop; the BLE manager and biometric aggregator handle wristband data; repositories are the only path to Firestore. In the cloud, Firebase AI Logic forwards chat requests to Gemini (dotted arrow: failover to the backup model), and the Vector Search extension finds relevant foods and exercises.

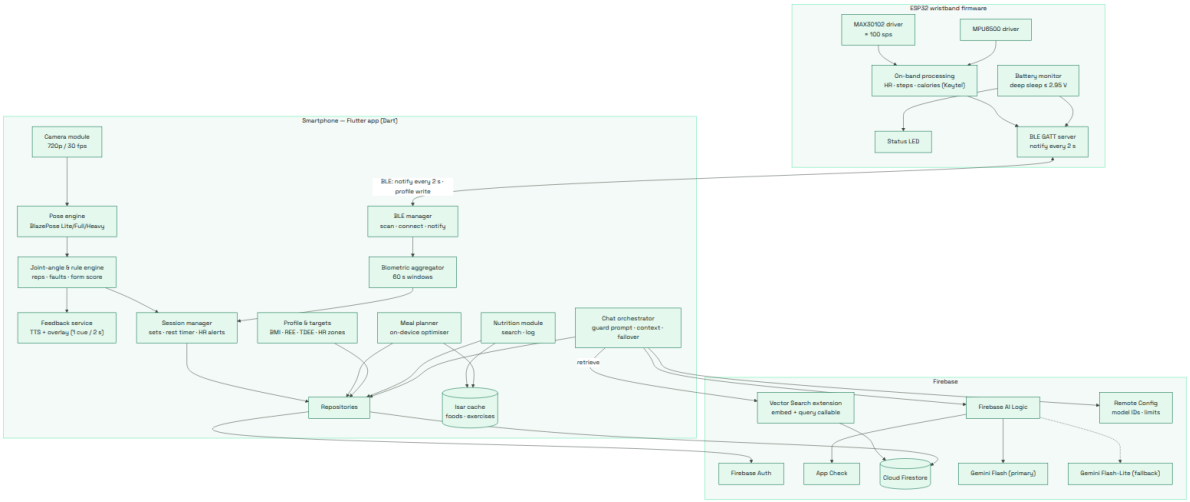


Figure 2: Component diagram

5. Deployment View

Figure 3 shows where each part physically runs: the wristband, the smartphone, the Firebase project, and Google's Gemini API (reached only through Firebase).

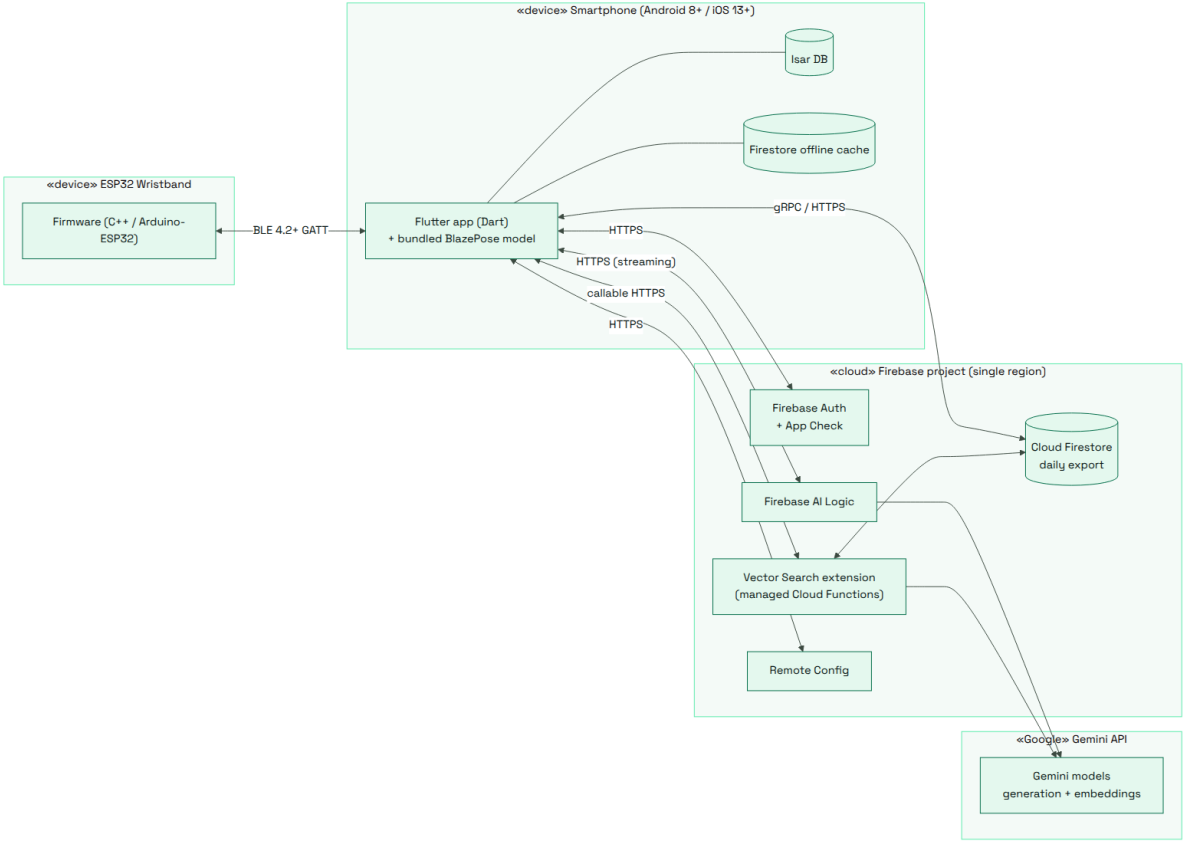


Figure 3: Deployment diagram

Why no custom backend: the team writes everything in Dart, and Firebase already provides every server-side job the system needs — key protection and rate limiting for the LLM (AI Logic), embeddings and vector search (extension), and access control (Security Rules). A separate server would add code, hosting cost and attack surface without adding capability.

6. Key Runtime Scenarios

6.1 Live Workout Set

The most performance-critical path. The camera → pose → rule → cue loop runs entirely on the phone, so the overlay updates within 100 ms and the spoken cue within 300 ms (NFR-PERF-02). Wristband data arrives in parallel every 2 s. Only when the set ends is anything written to Firestore — one batched write with the set summary.

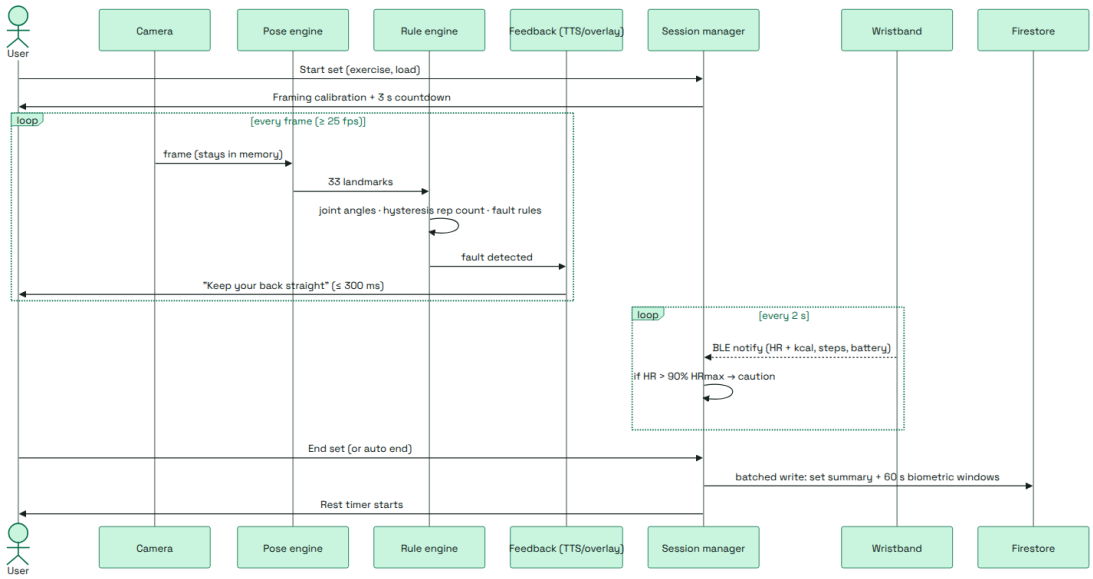


Figure 4: Live workout set

6.2 Chatbot Message with Failover

The app removes personal identifiers, retrieves relevant foods/exercises through vector search, and builds a guarded prompt with the user's goals and recent workout summary. It calls Gemini through Firebase AI Logic. If the primary model gives no first token within 5 s, the app retries with the backup model (NFR-REL-05); if both fail, a templated answer is shown.

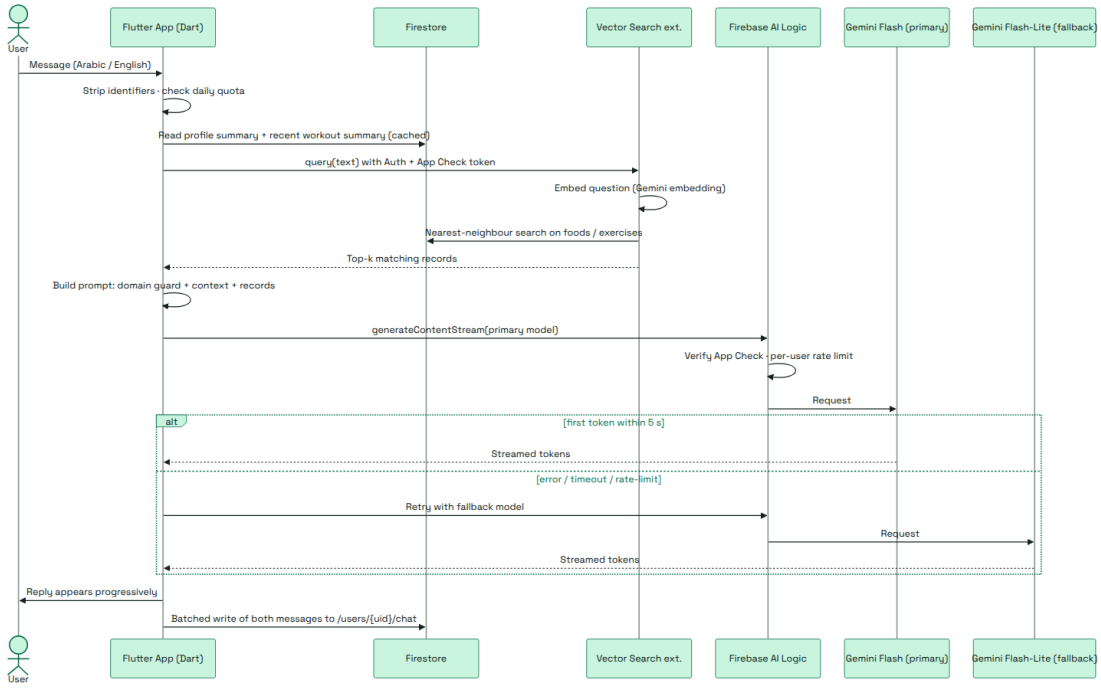


Figure 5: Chatbot message with provider failover

6.3 Offline Training and Reconnection

Every write is saved locally at once and synced automatically when connectivity returns (NFR-REL-02) [22]. The wristband keeps counting steps and calories while out of range and sends the correct totals on reconnect.

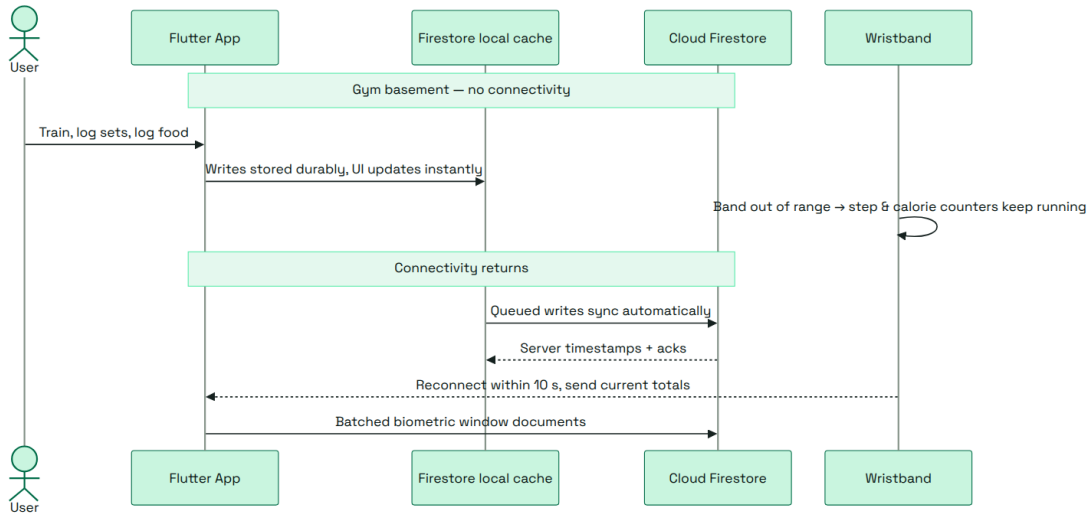


Figure 6: Offline training and reconnection

7. Pose-Coaching Design

7.1 Pipeline

1. **Capture** — 720p / 30 fps frames are processed in memory and discarded (FR-CV-11).
2. **Pose estimation** — BlazePose [2] returns 33 landmarks; low-visibility points are ignored and the user is asked to reposition if key joints are missing (FR-CV-09).
3. **Angles** — knee, hip, trunk, elbow and shoulder angles from 2-D vector maths (FR-CV-04).
4. **Rep counting** — a two-state machine with hysteresis: a rep counts only when the key angle goes below a lower threshold and back above an upper one, so jitter cannot double-count (FR-CV-05).
5. **Fault rules & feedback** — each rep is checked for knee valgus, depth, lumbar flexion, forward lean, elbow flare and asymmetry; the worst fault triggers one spoken + on-screen cue, at most every 2 s (FR-CV-06/07).
6. **Summary** — reps, time under tension, range of motion, tempo, form score 0–100 and fault counts (FR-CV-14).

7.2 Model Selection

Model	Latency [3]	Use
Lite	≈ 20 ms / frame	Mid-range phones; automatic fallback when frame rate drops
Full	≈ 25 ms / frame	Default on high-end phones
Heavy	≈ 53 ms / frame	Not used live — exceeds the 30 ms budget (NFR-PERF-01)

Table 4: Pose model tiers

Each exercise (squat, deadlift, bench press in v1) is described by **configuration data** — required landmarks, angle thresholds, fault rules and bilingual cues — stored in Firestore, so new exercises need no code change (NFR-MNT-03).

8. Wristband Design

Part	Role
ESP32	Microcontroller with built-in BLE; runs all on-band processing [5]
MAX30102	Optical heart-rate sensor, ≈ 100 samples/s while connected (NFR-PERF-11) [6]
MPU6500	Accelerometer/gyroscope for motion and step counting [7]
Status LED	Shows advertising, connected and low-battery states
Li-Po battery + voltage divider	Power and battery-level monitoring; target ≥ 8 h of session use (NFR-PWR-01)

Table 5: Wristband hardware

On-band processing. The band computes heart rate, steps and an estimated calorie burn itself, so only small values travel over BLE. Every 2 s it sends heart rate + calories, steps and battery level (NFR-PERF-04). It enters deep sleep if the battery stays at or below 2.95 V for 5 readings (NFR-PWR-06).

Calorie estimate. Calories are estimated from heart rate with the Keytel et al. (2005) equation [12], using the sex, age and weight the phone sends to the band on connection:

```
Male:   kcal/min = (-55.0969 + 0.6309*HR + 0.1988*W + 0.2017*A) / 4.184
Female: kcal/min = (-20.4022 + 0.4472*HR - 0.1263*W + 0.0740*A) / 4.184
```

BLE link. The band uses the standard Bluetooth Heart Rate service [8] (whose packet already carries an Energy Expended field, so heart rate and calories travel together) and Battery service [9], plus one custom service for steps, the user profile and commands. Steps and calories are cumulative counters, so totals are correct after a disconnection; the link re-establishes within 10 s (NFR-REL-04). The phone side uses the flutter_blue_plus package [11]; connection intervals follow BLE power guidance [10].

9. Cloud Services and AI

9.1 Firebase Instead of a Backend

Server-side job	Handled by
Sign-in, password storage and reset	Firebase Auth [19]
Proving requests come from the genuine app	App Check [18]
Data storage, offline sync, access control	Cloud Firestore + Security Rules [22], [25]
LLM access without exposing an API key; per-user rate limit	Firebase AI Logic (Gemini) [15], [17]
Embeddings and semantic search	Firestore Vector Search extension [23], [24]
Model names and limits changeable without an app release	Remote Config [20]

Table 6: Firebase services replacing a custom backend

9.2 Chatbot

The chatbot is assembled in the app from four layers: a **guard instruction** that keeps it to fitness, nutrition and safety and refuses medical diagnosis (FR-CHAT-03/06); the **user's context** — goal, experience, macro targets and a recent workout summary, never raw biometric data (FR-CHAT-04); **retrieved records** from vector search; and the **conversation**, answered in the user's language and dialect (FR-CHAT-01). Replies stream as they are generated (FR-CHAT-05). A Gemini Flash model is primary and Flash-Lite the fallback [16]; model names live in Remote Config so retired models can be swapped without an app update.

9.3 Semantic Search

Food and exercise documents are embedded with a multilingual Gemini embedding model by Firebase's Vector Search extension [24], so an Egyptian-Arabic question can match an English-named record. If search is unavailable or offline, the app falls back to its local fuzzy search.

9.4 Meal Planner

Plans are generated on the phone from the local catalogue, so they work offline: 3 meals and up to 2 snacks within ±5 % of TDEE and ±10 % of protein target (FR-NUT-05), within the user's budget in EGP (FR-NUT-06), and never containing an excluded food (FR-NUT-09).

10. Data Design

The full schema is delivered as a separate diagram (Firestore Schema Diagram). The collections are:

Collection	Contents
/users/{uid}	Profile, computed targets, preferences, consents
…/workouts/{id} → /sets/{id}	Workout sessions and per-set summaries (reps, form score, faults)
…/biometrics/{id}	Heart-rate, step and calorie windows (10-minute blocks)
…/meals/{date}	Daily food log, totals and generated plan
…/chat/{id}	Chat history (capped and pruned)
/foods, /exercises	Shared catalogues: Egyptian foods with EGP prices [28]; exercise configurations
/appConfig, /auditLog	App settings; append-only log of admin changes

Table 7: Firestore collections

10.1 Design Rules

- **Read together → store together:** one screen is served by as few reads as possible.
- **Growing data → subcollection:** workouts, sets, biometrics and chat never grow inside a single document (1 MiB limit, C-DATA-01).
- **Aggregate before writing:** biometric samples are never written one by one (C-DATA-02).
- **Security Rules deny by default:** users access only their own data; catalogues are read-only to users (FR-DATA-06).

10.2 Capacity Check Against the Free Tier

To support 500 daily users inside Firestore's free tier, each user can make about **40 writes per day** (NFR-SCAL-02) [21]. For a typical day with a 60-minute workout:

Activity	One doc per 60 s window	Ten windows per 10-min doc
Workout create + finish	2	2
Set summaries (15 sets)	15	15
Biometric documents (60 min)	60	6
Meal logging	6	6
Chat (5 questions)	10	10
Total	**93 — exceeds budget**	**39 — within budget**

Table 8: Daily writes per active user

Each biometric document therefore holds ten 60-second windows. The data is still aggregated per minute, but writes fall by 90 %.

11. Security, Privacy and Localisation

Concern	Design control
Passwords	Handled entirely by Firebase Auth; never stored in Firestore (FR-AUTH-04)
Camera data	Processed in memory and discarded; only set summaries leave the phone (NFR-SEC-04)
API keys	No Gemini key in the app — Firebase AI Logic holds it; App Check blocks modified apps (NFR-SEC-05)
Biometrics	Leave the phone only as aggregates, after consent; deleted after 12 months (NFR-SEC-07)
Medical boundary	“Wellness aid, not a medical device” notice; chatbot refuses diagnosis (NFR-LEG-01)

Table 9: Key security and privacy controls

Localisation. Arabic is a first-class feature, not a translation layer: the whole interface mirrors right-to-left in Arabic (FR-LOC-02), numbers, dates and prices follow the Egyptian locale (FR-LOC-03), and spoken coaching cues are written in natural Egyptian Arabic (FR-LOC-04), using Flutter’s localisation framework [27].

12. Key Design Decisions

#	Decision	Alternative	Why
D1	Phone camera as the only vision sensor	ESP32-CAM module	Free, flexible angle, already owned; ESP32-CAM adds cost and a fixed mount
D2	Pose AI runs on the device	Cloud vision API	Meets latency targets, works offline, no images leave the phone
D3	Firestore as the only database	Relational database	Required by C-TECH-02; built-in offline sync and security rules
D4	Dart app talks directly to Firebase	Custom Python / Node backend	One language for the team; Firebase covers every server job
D5	Gemini via Firebase AI Logic, with a fallback model	Groq via own backend	Keeps keys off the device with no server; automatic failover
D6	Biometrics stored in 10-minute block documents	One document per minute	Cuts writes by 90 % to stay in the free tier
D7	Exercises defined as configuration	Hard-coded logic per exercise	New exercises need no code change; thresholds tunable

Table 10: Key design decisions

13. Risks and Mitigations

Risk	Mitigation
Pose accuracy drops in dim light, at odd angles or with occlusion	Visibility filtering, repositioning prompt, framing calibration, tunable thresholds
Optical heart rate unreliable under sweat and motion	Low-confidence readings are flagged and skipped; validated against a chest strap (±10 bpm, NFR-DQ-01)
Wristband battery life below target	No display, moderate BLE interval, deep sleep below 2.95 V; measure current early
Gemini free-tier limits or model retirement	Fallback model, templated replies, model names in Remote Config
Firestore free-tier writes exceeded	Block biometric documents, batched writes, usage monitoring and budget alerts

Table 11: Main risks

14. References

[1] Spotter Team, "AI-Powered Smart Gym System: Requirements Engineering (SRS)," Alexandria National University, 2026. *(internal document)*

[2] V. Bazarevsky et al., "BlazePose: On-device Real-time Body Pose Tracking," CVPR Workshop, 2020. https://arxiv.org/abs/2006.10204

[3] Google, "MediaPipe Pose — solution documentation." https://mediapipe.readthedocs.io/en/latest/solutions/pose.html

[4] "pose_detection," Flutter package. https://pub.dev/packages/pose_detection

[5] Espressif Systems, "ESP32 Series Datasheet." https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[6] Analog Devices, "MAX30102 Datasheet." https://www.analog.com/media/en/technical-documentation/data-sheets/max30102.pdf

[7] TDK InvenSense, "MPU-6500 Product Specification." https://product.tdk.com/system/files/dam/doc/product/sensor/mortion-inertial/imu/data_sheet/mpu-6500-datasheet2.pdf

[8] Bluetooth SIG, "Heart Rate Service, v1.0." https://www.bluetooth.com/wp-content/uploads/Files/Specification/HTML/HRS_v1.0/out/en/index-en.html

[9] Bluetooth SIG, "Battery Service, v1.1." https://www.bluetooth.com/wp-content/uploads/Files/Specification/HTML/BAS_v1.1/out/en/index-en.html

[10] Silicon Labs, "Optimizing Current Consumption in Bluetooth Low Energy Devices." https://docs.silabs.com/bluetooth/2.13/bluetooth-general-system-and-performance/optimizing-current-consumption-in-bluetooth-low-energy-devices

[11] "flutter_blue_plus," Flutter package. https://pub.dev/packages/flutter_blue_plus

[12] L. R. Keytel et al., "Prediction of energy expenditure from heart rate monitoring during submaximal exercise," J. Sports Sci., vol. 23, no. 3, pp. 289–297, 2005. https://doi.org/10.1080/02640410470001730089

[13] M. D. Mifflin et al., "A new predictive equation for resting energy expenditure in healthy individuals," Am. J. Clin. Nutr., vol. 51, no. 2, pp. 241–247, 1990. https://doi.org/10.1093/ajcn/51.2.241

[14] H. Tanaka, K. D. Monahan and D. R. Seals, "Age-predicted maximal heart rate revisited," J. Am. Coll. Cardiol., vol. 37, no. 1, pp. 153–156, 2001. https://doi.org/10.1016/S0735-1097(00)01054-8

[15] Google, "Get started with the Gemini API using the Firebase AI Logic SDKs." https://firebase.google.com/docs/ai-logic/get-started

[16] Google, "Firebase AI Logic — supported models." https://firebase.google.com/docs/ai-logic/models

[17] "firebase_ai," Flutter package. https://pub.dev/packages/firebase_ai

[18] Google, "Firebase App Check." https://firebase.google.com/docs/app-check

[19] Google, "Firebase Authentication." https://firebase.google.com/docs/auth

[20] Google, "Firebase Remote Config." https://firebase.google.com/docs/remote-config

[21] Google, "Firestore — quotas and limits." https://cloud.google.com/firestore/quotas

[22] Google, "Firestore — access data offline." https://firebase.google.com/docs/firestore/manage-data/enable-offline

[23] Google, "Firestore — search with vector embeddings." https://firebase.google.com/docs/firestore/vector-search

[24] Google, "Vector Search with Firestore" (Firebase Extension). https://extensions.dev/extensions/googlecloud/firestore-vector-search

[25] Google, "Get started with Cloud Firestore Security Rules." https://firebase.google.com/docs/firestore/security/get-started

[26] Isar Database, "isar," Flutter package. https://pub.dev/packages/isar

[27] Flutter, "Internationalizing Flutter apps." https://docs.flutter.dev/ui/accessibility-and-internationalization/internationalization

[28] National Nutrition Institute, "Food Composition Tables for Egypt," via FAO / INFOODS. https://www.fao.org/infoods/infoods/tables-and-databases/egypt/en

Online sources accessed October 2026.

Appendix A. Traceability Matrix

Table 12: Traceability matrix

Requirements	Design element	Section
Account, Profile and Localisation		
FR-AUTH-01..07	Firebase Auth + /users/{uid} profile document	9.1, 10
FR-PROF-01..07	Profile & targets computed in the domain layer (BMI, TDEE, HR zones)	3.2
FR-LOC-01..05	RTL layout, Egyptian locale, Egyptian-Arabic cues	11
Pose Coaching		
FR-CV-01..11, FR-CV-14	On-device pose pipeline	7.1
FR-CV-03, FR-CV-13	Model tier selection and auto-downgrade	7.2
FR-CV-12, NFR-MNT-03	Exercises as configuration	7.2
NFR-PERF-01/02/10	Real-time loop kept on the phone	6.1
Workout Session and Alerts		
FR-ALRT-01..04	Session manager: sets, rest timer, HR caution, warm-up	4, 6.1
Wristband		
NFR-PERF-04/11	On-band HR, steps, calories; BLE updates every 2 s	8
NFR-PWR-01/06	Battery monitoring and deep sleep	8
NFR-REL-04, NFR-COMP-04, C-TECH-04	BLE link and reconnection	8
Nutrition and Chatbot		
FR-NUT-01..09	Egyptian food catalogue and on-device meal planner	9.4, 10
FR-CHAT-01..08	Chatbot via Firebase AI Logic with fallback model	6.2, 9.2
NFR-REL-05	Failover after 5 s without a first token	6.2
Data, Offline and Capacity		
FR-DATA-01..08, C-DATA-01..04	Firestore collection model and design rules	10, 10.1
FR-DATA-03, NFR-REL-02	Offline persistence and Isar catalogue cache	6.3
NFR-SCAL-02, NFR-PERF-12	Biometric block documents within free tier	10.2
Security, Privacy and Architecture		
NFR-SEC-01..12, NFR-LEG-01..05	Security and privacy controls	11
NFR-MNT-01/02, C-TECH-01..07	Three-layer Dart app + Firebase serverless architecture	3, 5, 9.1